

LOAN APPROVAL PREDICTION

A Supervised Machine Learning Analysis

By Yaksh Dhamat

Predicting Loan Outcomes from Borrower Features

Internship Task 2

614

Applications

5

Models Tested

0.846

Best ROC-AUC

10

Visualizations

Dataset: kaggle.com/datasets/bhanupratapbiswas/loan-approval-prediction-case-study

Tools: Python | pandas | scikit-learn | imbalanced-learn | seaborn

1. Executive Summary :

This report presents a supervised machine learning analysis to predict loan approval outcomes using borrower demographic, financial, and credit history features. The dataset contains 614 loan applications with a 68.7% / 31.3% class imbalance (approved vs rejected). The analysis covers data cleaning, exploratory analysis, feature engineering, leakage-safe class balancing with SMOTE, comparison of five classification models, and business-oriented threshold recommendations for deployment.

2. Dataset Overview :

2.1 Source & Description :

The dataset is sourced from Kaggle(<https://www.kaggle.com/datasets/bhanupratapbiswas/loan-approval-prediction-case-study>) and contains historical loan application records with approval outcomes.

Column	Type	Description
Loan_ID	Text	Unique application identifier
Gender	Category	Male / Female
Married	Category	Marital status (Yes/No)
Dependents	Category	Number of dependents (0, 1, 2, 3+)
Education	Category	Graduate / Not Graduate
Self_Employed	Category	Yes / No
ApplicantIncome	Numeric	Primary applicant's income
CoapplicantIncome	Numeric	Co-applicant's income
LoanAmount	Numeric	Requested loan amount (thousands)
Loan_Amount_Term	Numeric	Loan repayment term (months)
Credit_History	Binary	1 = good credit history, 0 = none/poor
Property_Area	Category	Urban / Semiurban / Rural
Loan_Status	Target	Y = Approved, N = Rejected

2.2 Data Quality Issues :

- Missing values across 7 columns: Gender (13), Married (3), Dependents (15), Self_Employed (32), LoanAmount (22), Loan_Amount_Term (14), Credit_History (50)
- Class imbalance: 68.7% approved vs 31.3% rejected
- Income and loan amount distributions are right-skewed

3. Methodology :

3.1 Pipeline Overview :

The analysis follows a strict, leakage-safe pipeline:

- **Step 1:** Load raw data and inspect structure (no visualizations yet)
- **Step 2:** Clean missing values using mode (categorical) / median (numeric) imputation
- **Step 3:** Run EDA only after cleaning - ensures charts and statistics reflect the true dataset
- **Step 4:** Engineer new features (Total_Income, log transforms, EMI, Balance_Income)
- **Step 5:** Encode categorical variables with Label Encoding
- **Step 6:** Split into train/test sets BEFORE balancing classes
- **Step 7:** Apply SMOTE only to the training set - test set remains untouched and imbalanced
- **Step 8:** Scale features, train 5 models, evaluate on the real-world imbalanced test set

3.2 Why Data Cleaning Must Precede EDA :

Running EDA on dirty data produces misleading charts and statistics - missing values distort category percentages, correlation calculations, and distribution shapes. All visualizations in this report were generated after imputation was complete.

3.3 Avoiding Data Leakage with SMOTE :

A critical methodological safeguard in this analysis: SMOTE (Synthetic Minority Oversampling Technique) was applied only to the training data, after the train/test split. Applying SMOTE before splitting would let synthetic data points borrow information from rows that later end up in the test set, artificially inflating accuracy and ROC-AUC. By splitting first, the test set remains a pure, untouched sample of real-world imbalanced data, producing trustworthy evaluation metrics.

4. Data Cleaning :

Column	Missing Count	Imputation Strategy
Gender	13	Mode (most frequent: Male)
Married	3	Mode (most frequent: Yes)
Dependents	15	Mode (most frequent: 0)
Self_Employed	32	Mode (most frequent: No)

Column	Missing Count	Imputation Strategy
LoanAmount	22	Median (robust to outliers)
Loan_Amount_Term	14	Mode (most frequent: 360)
Credit_History	50	Mode (most frequent: 1.0)

Result: All 614 rows retained — no rows dropped, since every record is valuable in a dataset of this size.

5. Exploratory Data Analysis :

5.1 Target Distribution :

Of 614 applications, 422 (68.7%) were approved and 192 (31.3%) were rejected — a meaningful class imbalance requiring correction before model training.

5.2 Categorical Features vs Approval

Feature	Observation
Married	Married applicants show a somewhat higher approval rate than unmarried
Education	Graduates show a marginally higher approval rate than non-graduates
Self_Employed	Minimal difference in approval rate between self-employed and salaried
Property_Area	Semiurban applicants show the highest approval rate among the three areas
Dependents	Applicants with 0 dependents show slightly higher approval rates

5.3 Credit History — The Dominant Factor :

The single most powerful insight in this dataset: applicants with a positive credit history are approved at a dramatically higher rate than those without.

Credit History	Approval Rate
Has History (1)	~80%
No History (0)	~8%

This single feature alone is nearly as predictive as the full model — see Feature Importance in Section 7.

5.4 Numeric Feature Distributions :

Approved loans show a slightly lower median loan amount than rejected ones. Applicant income distributions are heavily right-skewed for both outcomes, with most applicants earning under ₹10,000/month and a long tail of high earners.

5.5 Feature Correlations :

A correlation heatmap of cleaned and engineered features confirms Credit_History has by far the strongest correlation with Loan_Status. Income and loan-amount-derived features (Total_Income_log, EMI) show only weak correlation with the target — reinforcing that creditworthiness history, not income level, drives approval decisions in this dataset.

6. Feature Engineering :

New Feature	Formula	Rationale
Total_Income	$\text{ApplicantIncome} + \text{CoapplicantIncome}$	Household income is more relevant than individual income alone
LoanAmount_log	$\log(1 + \text{LoanAmount})$	Reduces right-skew for more stable model behaviour
Total_Income_log	$\log(1 + \text{Total_Income})$	Reduces right-skew for more stable model behaviour
EMI	$\text{LoanAmount} / \text{Loan_Amount_Term}$	Approximates monthly repayment burden
Balance_Income	$\text{Total_Income} - (\text{EMI} \times 1000)$	Estimates disposable income after loan repayment

7. Model Results :

7.1 Class Imbalance Handling :

Before SMOTE (training set only): 337 approved, 154 rejected. After SMOTE: 337 approved, 337 rejected — perfectly balanced. The test set (123 rows) was kept at its original imbalanced ratio to reflect real-world conditions.

7.2 Model Comparison — Evaluated on Untouched Test Set :

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	80.5%	82.4%	90.6%	0.890	0.846
Decision Tree	70.7%	78.1%	78.8%	0.785	0.708
Random Forest	77.2%	80.8%	87.1%	0.838	0.817
Gradient Boosting	75.6%	80.0%	84.7%	0.823	0.799
SVM	78.0%	80.6%	89.4%	0.847	0.836

Winner by ROC-AUC: Logistic Regression (0.846).

Logistic Regression achieves the best ROC-AUC on the honest, leakage-free test set. This is notable — simpler linear models often perform competitively on small tabular datasets (614 rows) where complex ensemble methods like Random Forest or Gradient Boosting risk overfitting to training noise rather than capturing genuinely generalizable patterns.

7.3 Feature Importance :

Feature importance from the Random Forest model confirms the EDA finding: Credit_History is the dominant predictor, far outweighing all income, demographic, and loan-amount features combined.

7.4 Confusion Matrix Interpretation :

The confusion matrix for the best-performing model (by F1 Score) shows the model correctly identifies the large majority of both approved and rejected applications, with most errors occurring in borderline-income, borderline-credit-history cases.

8. Business Interpretation & Threshold Analysis :

8.1 Threshold Trade-off Table :

Threshold	Precision	Recall	F1 Score	% Approved
0.3	76.9%	94.1%	0.846	76.4%
0.4	79.7%	92.9%	0.858	71.5%
0.5	82.4%	90.6%	0.890	65.0%
0.6	86.7%	76.5%	0.813	52.8%
0.7	90.0%	63.5%	0.745	41.5%

8.2 Recommended Deployment Threshold :

Threshold = 0.5 is recommended as the default deployment setting, offering the best balance between precision (82.4%) and recall (90.6%), with the highest F1 Score (0.890) of any threshold tested.

- **Recommendation 1:** Use threshold 0.5 (default) for balanced, general-purpose loan screening
- **Recommendation 2:** Use threshold 0.6–0.7 if the business prioritizes minimizing default risk over approval volume
- **Recommendation 3:** Use threshold 0.3–0.4 only during growth-focused periods where higher approval volume is prioritized over risk

8.3 Key Business Insight :

Credit_History is overwhelmingly the most important factor in loan approval, far outweighing income, education, or demographic features. This has two important implications for the business:

- The model largely mirrors decisions that credit bureaus already make — its incremental value lies in consistency and speed, not in discovering hidden patterns
- Applicants with no credit history ("thin-file" applicants) are rejected at a ~92% rate regardless of income — the business may want to explore alternative credit signals (utility payments, rent history) to fairly serve this segment

9. Recommendations :

#	Recommendation	Rationale
1	Deploy Logistic Regression as the production model	Best ROC-AUC (0.846) on honest test data; simple and interpretable
2	Use 0.5 as the default classification threshold	Best F1 Score balance between precision and recall
3	Always split data before applying SMOTE in future pipelines	Prevents data leakage and inflated performance metrics
4	Investigate alternative credit signals for thin-file applicants	Credit_History dominates; current model under-serves this segment
5	Re-evaluate threshold quarterly against actual default rates	Threshold should adapt as portfolio risk appetite changes

10. Future Scope :

- Collect alternative credit data (utility bills, rent payments) for thin-file applicants
- Hyperparameter tuning via GridSearchCV for Logistic Regression and SVM
- Test ensemble stacking of Logistic Regression with Random Forest for marginal gains
- Collect a larger dataset to reduce variance in model evaluation
- Monitor model performance over time for concept drift as economic conditions change

11. Tools & Methodology :

Tool / Library	Purpose
Python 3.10+	Core programming language
pandas / NumPy	Data loading, cleaning, manipulation
Matplotlib / Seaborn	Visualizations
scikit-learn	ML models, preprocessing, evaluation metrics
imbalanced-learn	SMOTE for class imbalance handling
Jupyter Notebook	Interactive development environment
VS Code	Primary code editor